



《AI大模型Ragent项目》——10个人同时提问只有3个坑位

来自：拿个offer-开源&项目实战



马丁

2026年05月14日 22:02

开篇引言

上一篇收尾了流式生成子系列，把单个请求的故事讲得比较透了——正常路径走完 `finish + done` 落库关连接，被取消路径走 `cancel + done` 保存部分内容再关连接，资源都能干净释放。镜头始终对着一个请求，从 LLM 第一个 Token 吐出到最后一个 Token 收束。

但是把镜头从单个请求拉远到整个集群，会看到完全不一样的压力面。

假设你在线教育公司做开发，公司前段时间上了 RAG 智能客服，主要做课程咨询、退费政策、报名引导这类问题。日常流量不大，平均下来一台机器同时跑 5 ~ 10 个请求。但每周三下午 3 点，运营会推一个「限时五折」的活动短信，全网用户瞬间涌进来问优惠详情、能不能叠加优惠券、报名截止时间。促销那一刻同时来了 200 个 RAG 请求。

这时候问题就来了：

下游模型服务有并发上限——本地部署的 GPU 显存有限，同时跑几十个推理任务直接 OOM；在线 API 平台（如 SiliconFlow）也有并发限制，一下涌进 200 个请求，绝大多数会直接报 429 Too Many Requests

Tomcat 默认 200 个工作线程会被这些 SSE 长连接占满——单个 RAG 请求要跑 30 秒到 1 分钟，普通的 REST 接口都进不来了

你可能会想：那加个限流就行了呗，不是几行代码的事？听起来很合理，但实际跑起来你会发现没那么简单——QPS 限流挡不住长连接、本地 Semaphore 在多机部署下管不住总并发、直接拒绝（HTTP 429）的体验又特别差。RAG 这种长耗时 + 资源敏感 + 体验敏感的场景，需要的是另一套思路：**排队**。

本篇聚焦排队限流的为什么 + 整体架构 + 入队与立即抢占的同步路径。下一篇接着讲异步等待路径——抢不到许可的请求怎么等、跨集群怎么唤醒、超时了怎么给用户交代。

为什么不能简单地「加个 QPS 限流」

要想清楚 RAG 的限流方案，得先想清楚 RAG 请求长什么样。

1. RAG 请求的三个特殊性

1.1 长耗时

普通 REST 接口几十毫秒到几百毫秒，QPS 限流非常合适——每秒最多放进 N 个请求，超过就拒绝，N 秒后再来。

但 RAG 是长连接，单次请求 10 秒到 1 分钟。考虑两种流量：

A 场景每秒来 10 个请求，每个跑 60 秒。第 1 秒进来 10 个开始跑；第 2 秒又来 10 个，但第 1 秒的还没跑完，同时在跑的变成 20 个；第 3 秒再来 10 个，堆到 30 个……一直堆积到第 60 秒，第 1 秒的 10 个刚好跑完退出，但第 2 秒到第 60 秒的还在跑——稳态并发 $10 \times 60 = 600$ 。

B 场景每秒也来 10 个请求，但每个只跑 100 毫秒。请求进来 100 毫秒就结束了，下一批还没到上一批就走了，任意时刻同时在跑的最多也就 1 个。

两者的 QPS 都是 10 req/s，QPS 限流看起来一视同仁。但 A 场景任意时刻有 600 个请求同时占着 GPU 显存和连接，B 场景只有 1 个。真正打爆下游的是「同时在跑的数量」，不是「每秒进来的数量」。

QPS 限流挡的是流量速率，挡不住并发存量。



下游 LLM 服务有硬性的并发上限。本地部署受 GPU 显存物理瓶颈限制——以一张 A100 跑 32B 模型为例，在常见量化和上下文配置下，单卡可承载的并发通常也就在几十量级，超出后要么排队导致延迟飙升，要么直接 OOM；在线 API 平台（如 SiliconFlow）同样设有 RPM/并发限制，超出会直接返回 429。不管是自建还是调三方，并发都有天花板，超了就是报错或拖垮延迟。

1.3 体验敏感

QPS 限流和 HTTP 429 是 REST 接口的常见组合——拒绝就拒绝了，前端弹一句「请稍后再试」，用户刷新一下重试。但 RAG 是用户已经按下回车并且看到了「思考中...」的加载圈圈。这时候直接告诉他「系统繁忙」，体验非常差——他会刷新再问，刷新再问，制造更多的请求把系统继续打爆。

2. 三种朴素方案的对比

针对上面三个特殊性，常见的三种方案各有适用场景。

方案	工作原理	适用场景	RAG 是否合适
直接拒绝 (HTTP 429)	超过并发立即返回错误码	短请求、用户能快速重试的场景	不合适，长连接 + 用户已等待，体验差
令牌桶 (Bucket4j)	按速率匀速发令牌，无令牌排队等待或拒绝	短请求 + 有突发但平均速率可控	不合适，令牌发出后长期占用，速率算不准
排队 + 信号量	总并发设上限，超出的入队等待，超时再拒绝	长耗时 + 资源敏感 + 体验敏感	合适，给等待者机会、给系统喘息空间

排队的核心思路是：先让请求进系统但不立刻执行，给前端一个「思考中...」的连接保持着。后台按 FIFO 顺序逐个抢许可，抢到就跑，没抢到就等，等不到就给个明确的反馈。这样既不会瞬时打爆下游，也不会粗暴拒绝用户。

3. 那用本地 Semaphore 行不行

学过并发的同学这时候会说：「我在 Java 进程里加一把 `java.util.concurrent.Semaphore`，构造时传 `new Semaphore(N)`，业务方法里 `acquire()` / `release()`，不就行了？」

如果你只部署一台机器，这套方案确实能工作。但 RAG 这种长耗时服务为了高可用一定是多机部署的，本地 Semaphore 在多机部署下立刻露馅：

节点 A 配置 Semaphore(N)，节点 B 也配置 Semaphore(N)，两台机器加起来集群总并发是 2N

你想限制集群总并发为 N，就得算 $N / \text{节点数}$ 配给每台机器；但节点数动态变化（后续的扩缩容），又得改配置

即使配对了，请求被负载均衡随机分发，节点 A 此刻满了节点 B 一个都没有，节点 B 的许可全在闲置，节点 A 后来的请求却被本地拒绝

本地 Semaphore 各算各的，没有全局视角。要想让整个集群按一个统一的并发上限执行，许可必须放在集群共享的地方——Redis 是最自然的选择。

Redisson 中的 `RPermitExpirableSemaphore` 把许可放在 Redis 里集中维护，集群里任何一个节点 `tryAcquire()` 都从同一个池子里取。这样集群总并发才真正被卡住。

整体架构：五个 Redis 数据结构 + 两层本地组件

理解完为什么需要排队，现在来看整体架构。本篇的限流系统由五个 Redis 数据结构和两层本地组件构成，各司其职。

1. 一张图看完整结构



整体看来分四段：

1. Tomcat 接收 SSE 请求，RAGChatServiceImpl.streamChat() 直接调用 ChatQueueLimiter.enqueue();
2. ChatQueueLimiter 是一个轻量的 SSE 业务编排层，它不直接操作 Redis，而是把排队抢占委托给底层的 FairDistributedRateLimiter——一个通用的分布式公平限流器；
3. FairDistributedRateLimiter 操作 Redis 五件套——信号量 + ZSET 队列 + 自增 seq + entry 存活标记 + Pub/Sub；
4. 抢到许可后，业务在专用线程池里执行，Tomcat 线程立刻返回。

为什么要拆两层？因为排队抢占的逻辑是通用的——信号量 + ZSET + Lua 原子 claim + entry 存活标记 + Pub/Sub 广播，跟 SSE、跟聊天业务没有半毛钱关系。把这套逻辑收拢到 FairDistributedRateLimiter，上层不管是 Chat 限流、文档导入限流还是其他场景，都能直接复用。ChatQueueLimiter 只负责 SSE 特有的事情：把 SseEmitter 的生命周期绑定到排队取消、超时被拒时写会话记忆和推 SSE 事件。

2. 五个数据结构各司其职

2.1 RPermitExpirableSemaphore：集群总并发上限

第一个数据结构是 Redisson 提供的可过期信号量。它持有 N 个许可（N 由 max-concurrent 配置），代表整个集群同时能跑的最大 RAG 任务数。

为什么用「可过期」版本而不是普通 RSemaphore？后面会单独开一节讲。这里只点一句：可过期版本的许可绑定 lease（租约），如果调用方崩溃没释放，租约到期后 Redis 自动回收，避免许可永久泄漏。

2.2 ZSET 队列：等待者按 FIFO 排序

第二个数据结构是 ZSET（有序集合），用来存所有正在等待许可的请求。每个等待者用 requestId 作为成员，用一个单调递增的 seq 作为 score。score 越小排名越前，最早入队的排在最前面。

为什么不用 List？后面会有专门的对比表。这里也点一句：ZSET 支持 ZRANK 查任意成员的位置、支持 ZREM 删任意成员，恰好对应「队头窗口判断」和「请求随时取消」这两个核心需求。

2.3 自增 seq：score 不冲突

第三个数据结构是 RAtomicLong，用 incrementAndGet() 生成全局单调递增的 seq 作为 ZSET 的 score。

你可能会想：直接用 System.currentTimeMillis() 当 score 不就行了吗？看起来很合理，但实际跑起来你会发现毫秒级冲突非常常见——同一毫秒内进来的多个请求 score 完全相同，ZSET 对同 score 元素按 lex 顺序排，结果 FIFO 语义被破坏，后入队的可能反而排在前面。单调 seq 永远不会冲突，ZRANK 给出的排名就是真正的入队顺序。

2.4 Entry 存活标记 (RBucket + TTL)：防僵尸条目

第四个数据结构是带 TTL 的 RBucket（普通 KV），每个入队的请求在写 ZSET 之前先写一个 rag:global:chat:entry:{requestId} 的 key，值无所谓（固定写 "1"），关键是设了一个 TTL——等待预算 + 5 秒缓冲。

为什么需要这个标记？考虑一个场景：节点 A 入队后立刻崩溃（进程被 kill），ZSET 里的条目没人清理，它永久占据队头窗口，排在它后面的请求永远无法 claim。entry 标记就是这个问题的兜底——JVM 崩溃后 key 自然过期，后续的 Lua claim 脚本在扫描队头窗口时会检查每个成员的 entry 标记是否存在，不存在的视为僵尸条目直接 ZREM，把队头让给存活的请求。

简单理解：ZSET 存的是排队凭证，entry 标记存的是排队者还活着的证明。凭证还在但人已经不在，就是僵尸。

2.5 RTopic Pub/Sub：跨集群广播

第五个是 Pub/Sub 频道，用来在「许可释放」时广播通知所有节点。节点 A 上的请求释放许可后，节点 B 上等待的请求需要被唤醒来重新抢——单靠节点 B 自己轮询太慢，靠 Pub/Sub 立刻通知最高效。
广播的具体机制和上一篇 StreamTaskManager 的取消广播是同一套套路。本篇不展开，下篇会详细讲跨集群唤醒和 PollNotifier 防惊群。

3. 配置参数一览

把抽象的架构落到具体的配置参数上：

```
rag:
  rate-limit:
    global:
      enabled: true
      max-concurrent: 3      # 演示用 3 个许可
      max-wait-seconds: 20
      lease-seconds: 600
      poll-interval-ms: 200
```

参数	含义	经验值
enabled	是否启用全局限流	生产开，本地 dev 可关
max-concurrent	集群总并发上限（信号量许可数）	按下游 LLM 并发上限设，本地 GPU 看显存，在线 API 看平台限制
max-wait-seconds	单个请求最长等待时间	20 ~ 30 秒，超过用户体验已不可接受
lease-seconds	许可租约（兜底自动回收）	600 秒 = 10 分钟，比业务最长耗时大一些
poll-interval-ms	等待者轮询间隔	200 ms，下篇详细讲

为了让后文的场景描述更具体，咱们把这家长在线教育公司的配置定下来：max-concurrent=3，部署两台机器。促销时刻同时来了 10 个用户提问。本篇要回答的问题是：前 3 个请求怎么抢到许可？如果只剩 1 个许可而 10 个请求同时来抢，谁能抢到？

限流入口：streamChat 直接委托 ChatQueueLimiter

先讲入口设计，因为它决定了业务代码长什么样。

1. 调用方：RAGChatServiceImpl

```

@Override
public void streamChat(String question, String conversationId, Boolean deepThinking, SseEmitter emitter,
    String actualConversationId = StrUtil.isBlank(conversationId) ? IdUtil.getSnowflakeNextIdStr()
    String taskId = IdUtil.getSnowflakeNextIdStr();
    StreamCallback callback = callbackFactory.createChatEventHandler(emitter, actualConversationId

    StreamChatContext ctx = StreamChatContext.builder()
        .question(question)
        .conversationId(actualConversationId)
        .taskId(taskId)
        .deepThinking(Boolean.TRUE.equals(deepThinking))
        .userId(UserContext.getUserId())
        .callback(callback)
        .build();

    chatQueueLimiter.enqueue(question, actualConversationId, emitter,
        () -> traceRunner.run(question, actualConversationId, taskId, callback,
            () -> chatPipeline.execute(ctx)));
}

```

streamChat() 做的事情拆开看：

补齐 conversationId：如果调用方没传（首轮对话），用 snowflake 生成一个，确保后续能正确入库

组装 StreamChatContext：把问题、会话 ID、任务 ID、用户 ID 等打包

调用 chatQueueLimiter.enqueue()：把真正的业务执行包装成一个 lambda 交给排队器

最后一步是关键——把 chatPipeline.execute(ctx) 包成 lambda，外面再裹一层 traceRunner.run()（全链路追踪埋点），整体作为 onAcquire 回调传给 enqueue()。业务方法本身不会在调用线程中同步执行。什么时候真正执行？等 enqueue() 抢到许可后，在专用线程池里跑那个 lambda 时才执行。

traceRunner 是 StreamChatTraceRunner，内部把每次执行的状态写到全链路追踪表里。Trace 不是本篇主题，简单理解为带追踪的方法调用即可。

2. ChatQueueLimiter.enqueue()

enqueue() 的方法签名值得仔细看：

```

public void enqueue(String question, String conversationId, SseEmitter emitter, Runnable onAcquire

```

四个参数：

question：用户问题，主要用于超时拒绝时把消息写入会话记忆（下篇主题）

conversationId：会话 ID，同上

emitter：SSE 发送器，排队过程中可能要推事件、关连接

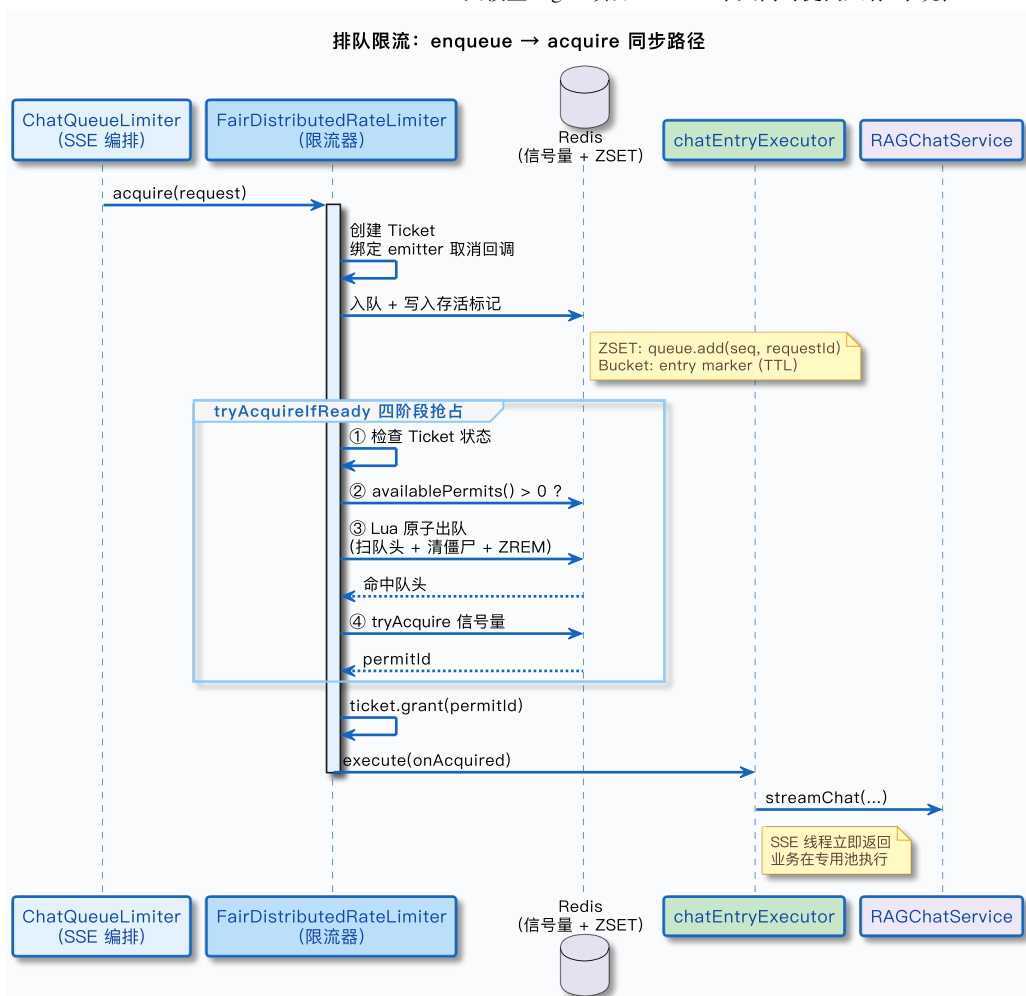
onAcquire：抢到许可后真正执行业务的回调

onAcquire 是整个设计的关键——它把「业务执行」这个动作变成了一个可延迟、可异步、可丢给线程池的可执行对象。streamChat() 不直接跑业务，而是把业务包成 lambda 交给排队器，由排队器决定什么时候、在哪个线程上执行。业务代码看起来还是直接执行，实际中间插了一层「排队 → 抢占 → 异步执行」。

enqueue()：入队 + 立即抢占的同步路径

到了本篇真正的核心。enqueue() 是同步路径——请求进来后，要么立刻抢到许可走快速通道，要么进入异步等待路径（下篇主题）。本节聚焦快速通道。

1. 完整流程图



把流程顺下来：进入 ChatQueueLimiter.enqueue() → 检查总开关 → 构建 AcquireRequest 委托给

FairDistributedRateLimiter.acquire() → 创建 Ticket 并绑定 emitter 取消回调 → 写入 entry 存活标记 → 写入 ZSET 占位 → 调用 tryAcquireIfReady() 四阶段抢占。如果四阶段全部成功，ticket.grant() 提交线程池跑业务；如果中间某一步失败，进入下篇的异步等待路径。

先看 ChatQueueLimiter.enqueue() 的完整代码——它非常短，因为所有排队抢占逻辑都委托给了 FairDistributedRateLimiter：

```

public void enqueue(String question, String conversationId, SseEmitter emitter, Runnable onAcquire) {
    if (!Boolean.TRUE.equals(rateLimitProperties.getGlobalEnabled())) {
        try {
            chatEntryExecutor.execute(onAcquire);
        } catch (RejectedExecutionException ex) {
            log.warn("直通分支线程池拒绝任务，转 reject 流程", ex);
            handleReject(question, conversationId, emitter);
        }
        return;
    }

    chatRateLimiter.acquire(AcquireRequest.builder()
        .maxWaitMillis(TimeUnit.SECONDS.toMillis(rateLimitProperties.getGlobalMaxWaitSeconds()))
        .onAcquired(onAcquire)
        .onTimeout(() -> handleReject(question, conversationId, emitter))
        .onAcquiredExecutor(chatEntryExecutor)
        .cancelBinder(cancel -> {
            emitter.onCompletion(cancel);
            emitter.onTimeout(cancel);
            emitter.onError(e -> cancel.run());
        })
        .build());
}

```

chatRateLimiter 就是 FairDistributedRateLimiter 的实例，通过 Spring 注入。真正的入队、抢占、异步等待全在 acquire() 内部完成。ChatQueueLimiter 只做三件事：检查总开关、组装 AcquireRequest、提供 SSE 相关的回调。

再看 FairDistributedRateLimiter.acquire() 的完整代码：

```
public void acquire(AcquireRequest req) {
    Ticket ticket = new Ticket(req);
    if (req.cancelBinder() != null) {
        req.cancelBinder().accept(ticket::cancel);
    }
    setEntryMarker(ticket.requestId, req.maxWaitMillis());
    RScoredSortedSet<String> queue = redissonClient.getScoredSortedSet(queueKey, StringCodec.INSTA
    queue.add(nextQueueSeq(), ticket.requestId);
    if (tryAcquireIfReady(ticket)) {
        return;
    }
    scheduleQueuePoll(ticket);
}
```

这才是真正干活的方法。下面分块讲。

2. 配置开关：可禁用

```
if (!Boolean.TRUE.equals(rateLimitProperties.getGlobalEnabled())) {
    try {
        chatEntryExecutor.execute(onAcquire);
    } catch (RejectedExecutionException ex) {
        log.warn("直通分支线程池拒绝任务，转 reject 流程", ex);
        handleReject(question, conversationId, emitter);
    }
    return;
}
```

第一行就是总开关。某些环境（本地 dev）可能不希望走 Redis——本地起 Redis 麻烦、调试也不需要限流。这时候配置 enabled：false，请求直接走专用线程池，跳过整个排队流程。

注意即使关闭限流也走 chatEntryExecutor.execute(onAcquire) 而不是直接执行 onAcquire.run()——保持「业务在专用池里跑」这个不变量，让 SSE 接收线程能尽快返回，避免阻塞 Tomcat 线程池。

这里还多了一层 try-catch：如果线程池满了（SynchronousQueue 无缓冲，没有空闲线程就直接拒绝），走 handleReject 给用户推系统繁忙消息，而不是让异常上抛导致 SSE 连接无响应挂死。

线程池拒绝策略可以使用当前，也可以改为提交任务线程运行，具体看企业个性化需求。

3. 创建 Ticket + 入队

```
Ticket ticket = new Ticket(req);
if (req.cancelBinder() != null) {
    req.cancelBinder().accept(ticket::cancel);
}
setEntryMarker(ticket.requestId, req.maxWaitMillis());
RScoredSortedSet<String> queue = redissonClient.getScoredSortedSet(queueKey, StringCodec.INSTANCE)
queue.add(nextQueueSeq(), ticket.requestId);
```

acquire() 进来第一件事是创建一个 Ticket——它是整个排队流程的状态载体，后面的抢占、取消、超时都围着它转（下一小节详细讲）。cancelBinder 把外部的取消信号绑定到 ticket::cancel——在 Chat 场景里就是把 SseEmitter 的三个回调绑上去。然后是入队的三个动作。

3.1 setEntryMarker：先写存活标记再入队

```
setEntryMarker(ticket.requestId, req.maxWaitMillis());
```

在写 ZSET 之前，先写一个 `rag:global:chat:entry:{requestId}` 的 Redis key，TTL = 等待预算 + 5 秒缓冲。这个顺序不能反——如果先入队再写标记，存在一个极短的时间窗口：其他节点的 Lua claim 脚本扫描到这个 ZSET 条目时还看不到对应的 entry key，会把它当僵尸条目 ZREM 掉。先写标记再入队，保证 Lua 扫描时一定能看到存活证明。

3.2 requestId 用 snowflake

Ticket 构造时自动用 snowflake 生成 requestId，全局唯一且递增。不依赖 IP 不依赖用户 ID——同一个用户连续提问可以并发存在多个 requestId，互相独立。

3.3 nextQueueSeq() 用 RAtomicLong

```
private long nextQueueSeq() {  
    RAtomicLong seq = redissonClient.getAtomicLong(queueSeqKey);  
    return seq.incrementAndGet();  
}
```

这是 ZSET 的 score 来源。每次入队都从 Redis 拿一个递增数字，全集群共享一个计数器，保证所有节点入队的 seq 单调递增不冲突。为什么不用 `System.currentTimeMillis()`？前面已经提到过——毫秒级冲突会破坏 FIFO。在线教育公司促销那一刻 10 个请求几乎同时打到不同节点，时间戳完全可能撞车，但 `incrementAndGet()` 永远不会。

3.4 queue.add(nextQueueSeq(), requestId) 写 ZSET

把 (seq, requestId) 写进 ZSET。score 越小排名越前，所以最早入队的 requestId 排名为 0，第二个排名为 1，以此类推。到这里，10 个请求同时进来，节点 A 上的 5 个 + 节点 B 上的 5 个，每个调一次 `nextQueueSeq()`，每个 `queue.add()`，最终 Redis 的 ZSET 里有 10 个成员，按入队的真实顺序排好。注意此时还没有任何一个请求拿到许可，大家都在排队。

4. Ticket 状态机 + cancelBinder 绑定取消

老版本用一个 Runnable `releaseOnce` 手动管理 `cancelled` 标志位 + `permitRef` 原子引用。重构后引入了 Ticket 状态机，把「取消」「超时」「成功」三条路径的竞态处理统一收拢。先看 Ticket 的核心结构：


```
private enum State { PENDING, GRANTED, TIMED_OUT, CANCELLED }

private final class Ticket {
    final String requestId = IdUtil.getSnowflakeNextIdStr();
    final long deadline;
    final AcquireRequest req;
    final AtomicReference<State> state = new AtomicReference<>(State.PENDING);
    final AtomicReference<String> permitRef = new AtomicReference<>();
    volatile ScheduledFuture<?> future;

    void cancel() {
        state.compareAndSet(State.PENDING, State.CANCELLED);
        cleanup();
    }

    void timeout() {
        if (!state.compareAndSet(State.PENDING, State.TIMED_OUT)) {
            return;
        }
        cleanup();
        submitSafely(req.onTimeout(), "onTimeout");
    }

    boolean grant(String permitId) { /* ... 下文详解 ... */ }

    void cleanup() {
        // 移除队列 + 删除 entry 标记 + 释放 permit (非 GRANTED 状态下)
        // + 注销 poller + 取消 future, 所有子操作各自幂等
    }
}
```

一个 Ticket 从 PENDING 出发，只能**单次 CAS** 转到三个终态之一——GRANTED（抢到许可）、TIMED_OUT（超时被拒）、CANCELLED（外部取消）。CAS 保证不管多少个线程同时试图转换状态，只有一个能成功，业务回调最多触发一次。

接下来看取消绑定。在 ChatQueueLimiter.enqueue() 里，cancelBinder 把 ticket::cancel 绑定到 SseEmitter 的三个回调：

```
.cancelBinder(cancel -> {
    emitter.onCompletion(cancel);
    emitter.onTimeout(cancel);
    emitter.onError(e -> cancel.run());
})
```

这里的 cancel 就是 ticket::cancel。Ticket.cancel() 做两件事：

state.compareAndSet(State.PENDING, State.CANCELLED)：如果还在排队（PENDING），CAS 抢占终态，阻止后续 grant() 或 timeout() 的回调执行

cleanup()：幂等清理——从 ZSET 移除自己、删除 entry 存活标记、释放已持有的许可（**仅在非 GRANTED 状态下**）、注销 poller、取消定时任务

为什么 cleanup() 在 GRANTED 状态下不释放许可？因为一旦 grant() CAS 成功拿到 GRANTED 终态，许可的生命周期就由 grant() 内部的 try/finally 包装接管了——业务执行完毕（含异常）会在 finally 里调 releaseHeldPermit() 释放。如果 cancel() 的 cleanup() 也去释放许可，就会在业务还在跑的时候把许可还给信号量，等于让另一个请求拿到同一个 slot，突破并发上限。

那如果 grant() CAS 成功但还没来得及提交线程池就崩了呢？不用担心——grant() 里 execute() 抛 RejectedExecutionException 时会显式调 releaseHeldPermit() 释放许可，不会泄漏。

细心的读者可能注意到 onError 的写法和另外两个不一样——onCompletion 和 onTimeout 直接传 cancel (Runnable)，而 onError 包了一层 e -> cancel.run()。这是因为 onError 的参数类型是 Consumer<Throwable> 而不是 Runnable，需要用 lambda 适配一下。这里我们不关心具体的异常对象，所以直接忽略参数 e，只跑清理逻辑。

5. 立即抢占：跳过排队的快速路径

```
if (tryAcquireIfReady(ticket)) {  
    return;  
}  
scheduleQueuePoll(ticket);
```

入队 + 绑定取消后，立刻尝试抢占许可。这一步是优化，目的是让空闲时刻进来的请求不进调度循环。

回到在线教育公司的场景：平时晚上 11 点没什么人，许可 3 个全空。这时候有个学生想问明天报名截止时间，请求进来 → 入队（自己排第 0 名）→ tryAcquireIfReady 立刻看到队头窗口空着 + 信号量有许可，一把抢到走人。整个过程加起来不到 50 毫秒，根本不需要进异步等待路径。

注意现在所有参数都收拢在 Ticket 一个对象里——不再像老版本那样把 queue、requestId、permitRef、cancelled、onAcquire 五个参数一个个传进去。Ticket 既是状态载体又是参数包，方法签名干净很多。

只有在 tryAcquireIfReady 返回 false 时（许可没了 / claim 失败 / 抢到许可后又取消等），才会调 scheduleQueuePoll 进入异步等待。这部分是下篇主题，本篇就此打住。

tryAcquireIfReady(): 四阶段抢占

tryAcquireIfReady 是排队限流的灵魂方法，现在住在 FairDistributedRateLimiter 里。先看完整代码：

```
private boolean tryAcquireIfReady(Ticket ticket) {  
    if (!ticket.isPending()) { // 阶段 1  
        return false;  
    }  
    int avail = availablePermits(); // 阶段 2  
    if (avail <= 0) {  
        return false;  
    }  
    long claimedScore = claimIfReady(ticket.requestId, avail); // 阶段 3  
    if (claimedScore < 0L) {  
        return false;  
    }  
    String permitId = tryAcquirePermit(); // 阶段 4  
    if (permitId == null) {  
        setEntryMarker(ticket.requestId,  
            Math.max(1, ticket.deadline - System.currentTimeMillis()));  
        RScoredSortedSet<String> queue = redissonClient  
            .getScoredSortedSet(queueKey, StringCodec.INSTANCE);  
        queue.add(claimedScore, ticket.requestId);  
        publishQueueNotify();  
        if (!ticket.isPending()) {  
            queue.remove(ticket.requestId);  
            deleteEntryMarker(ticket.requestId);  
        }  
        return false;  
    }  
    if (!ticket.isPending()) { // 阶段后处理：再查状态  
        releasePermitQuietly(permitId);  
        publishQueueNotify();  
        return false;  
    }  
    publishQueueNotify();  
    return ticket.grant(permitId); // 提交线程池  
}
```

四个阶段每一个都不能省略，下面逐一拆解。

1. 阶段 1：状态检查

```
if (!ticket.isPending()) {
    return false;
}
```

进入方法第一件事就是查 Ticket 状态。isPending() 内部就是 state.get() == State.PENDING——只有还在排队的请求才有必要继续抢占。

考虑一个时序：请求 A 刚 queue.add() 入队，还没来得及调 tryAcquireIfReady，前端用户突然关浏览器了。emitter.onCompletion 被触发，ticket.cancel() 跑完，状态已经从 PENDING 变成 CANCELLED，队列里的 requestId 也被 cleanup() 移除了。这时候我们再走进 tryAcquireIfReady，如果不查状态，就会去 Lua 脚本 claim 一个不存在的请求 ID（被 ZREM 移除过）—— Lua 会返回 {0}，浪费一次 Redis RTT；更糟的是如果这个 ID 因为某种原因还在队列里，我们会把许可发给一个已经不存在的连接。

阶段 1 的存在是为了最早的短路。

2. 阶段 2：availablePermits 快速判断

```
int avail = availablePermits();
if (avail <= 0) {
    return false;
}
```

第二阶段查信号量当前可用许可数。如果是 0，直接返回 false，跳过后面的 Lua 脚本。

为什么要单独多查一次许可数？这一步是优化——availablePermits() 是一次轻量的 Redis 查询，而 Lua 脚本要发整段脚本到 Redis 执行 ZRANGE + EXISTS + ZREM，开销更大。当并发高峰期许可全占满时，绝大多数请求都会在阶段 2 被挡住，不需要进 Lua。

回到在线教育公司的演示场景：促销时刻 10 个请求涌进来，前 3 个抢到许可后，剩下 7 个进来调 tryAcquireIfReady，阶段 2 一查 availablePermits = 0，立刻返回 false，跳到 scheduleQueuePoll 走异步等待路径。生产环境请求量可能是几百个，阶段 2 能帮 Redis 省掉大量无谓的 Lua 调用。

这里会有一个理论上的窗口：阶段 2 查到 availablePermits > 0，但等到阶段 4 真去 tryAcquire 时许可被别人抢走了。tryAcquire 会返回 null，触发重入队 + 广播的兜底（阶段 4 的代码会讲）。所以阶段 2 是优化不是判定，最终判定还是看阶段 4。

3. 阶段 3：Lua 脚本 claim 队列位置

第三阶段是 FIFO 公平性的关键。

3.1 claimIfReady 调用 Lua

```
private long claimIfReady(String requestId, int availablePermits) {
    RScript script = redissonClient.getScript(StringCodec.INSTANCE);
    List<Object> result = script.eval(
        RScript.Mode.READ_WRITE,
        claimLua,
        RScript.ReturnType.LIST,
        List.of(queueKey),
        requestId,
        String.valueOf(availablePermits),
        entryKeyPrefix
    );
    if (result == null || result.isEmpty() || parseLong(result.get(0)) != 1L) {
        return -1L;
    }
    return result.size() >= 2 ? parseLong(result.get(1)) : nextQueueSeq();
}
```

Java 这边的工作很简单——把 ZSET 的 key、当前 requestId、当前可用许可数、entry 存活标记 key 前缀四个参数传给 Lua 脚本，让它在 Redis 端原子执行。返回值是一个数组：第一位是 0 / 1 表示是否 claim 成功，第二位是原始 score。Java 侧把它收敛成一个 long——成功返回原始 score（重入队时保留位次），失败返回 -1。

3.2 Lua 脚本逐行解读

```
-- 当请求位于允许的队头窗口内时，进行出队 claim；同时清理过期僵尸条目
-- KEYS[1]：队列 ZSET Key
```

```

-- ARGV[1]: 请求 ID
-- ARGV[2]: 最大可进入的 rank (可用许可数)
-- ARGV[3]: entry 存活标记 Key 前缀 (Java 侧已 set with TTL, 缺失即视为僵尸)
local queueKey = KEYS[1]
local requestId = ARGV[1]
local maxRank = tonumber(ARGV[2])
local entryPrefix = ARGV[3]

-- 取头部窗口 + 额外 slack: slack 用于在僵尸密集时尽量推进存活条目至 maxRank 之内
local slack = 16
local headEntries = redis.call('ZRANGE', queueKey, 0, maxRank + slack - 1)

local liveRank = -1
local liveCount = 0
for i = 1, #headEntries do
    local member = headEntries[i]
    if redis.call('EXISTS', entryPrefix .. member) == 1 then
        if member == requestId then
            liveRank = liveCount
        end
        liveCount = liveCount + 1
    else
        -- 僵尸 (Java 侧 entry 标记已 TTL 过期或被显式删除), 从队列移除
        redis.call('ZREM', queueKey, member)
    end
end

-- 不在存活队头窗口内 (要么不在队列、要么落在 maxRank 之外)
if liveRank < 0 or liveRank >= maxRank then return {0} end

-- 获取原始 score (便于必要时按原位次重入队)
local score = redis.call('ZSCORE', queueKey, requestId)

-- 出队 claim: 同步删除 entry 标记, 避免后续被自己误判
redis.call('ZREM', queueKey, requestId)
redis.call('DEL', entryPrefix .. requestId)

return {1, score}

```

逐行讲：

KEYS[1] 是 ZSET 队列的 Redis key (rag:global:chat:queue)。Redis 集群模式下，KEYS 列出的所有 key 必须在同一个 slot，Redisson 调用时会按这个约定走

ARGV[1] 是当前请求的 ID，ARGV[2] 是最大可 claim 的 rank (即可用许可数)，ARGV[3] 是 entry 存活标记的 key 前缀 (如 rag:global:chat:entry:)

ZRANGE queueKey 0 maxRank+slack-1 取出队头窗口 + 额外 16 个 slack 成员。为什么不直接取前 maxRank 个？因为队头可能有僵尸条目 (进程崩溃后 entry key 已过期)，如果只取 maxRank 个全是僵尸，存活请求就永远 claim 不到。多取 16 个 slack 给了足够的缓冲——即使队头有大量僵尸，也能让后面的存活条目推进到窗口内

接下来遍历 headEntries，对每个成员检查 EXISTS entryPrefix..member。如果 entry 标记存在说明请求还活着，计入 liveCount；如果不存在说明是僵尸，直接 ZREM 清理掉。遍历过程中如果遇到当前 requestId，记录它在存活成员中的排名 liveRank

if liveRank < 0 or liveRank >= maxRank then return {0} end: liveRank < 0 说明当前请求不在队列里 (被取消移除了)；liveRank >= maxRank 说明在存活成员中排名不够靠前。两种情况都 claim 失败

ZSCORE queueKey requestId 取原始 score——为了 claim 成功但 acquire 失败时能按原 score 重新入队，保留原来的排队位次，维护 FIFO 公平性

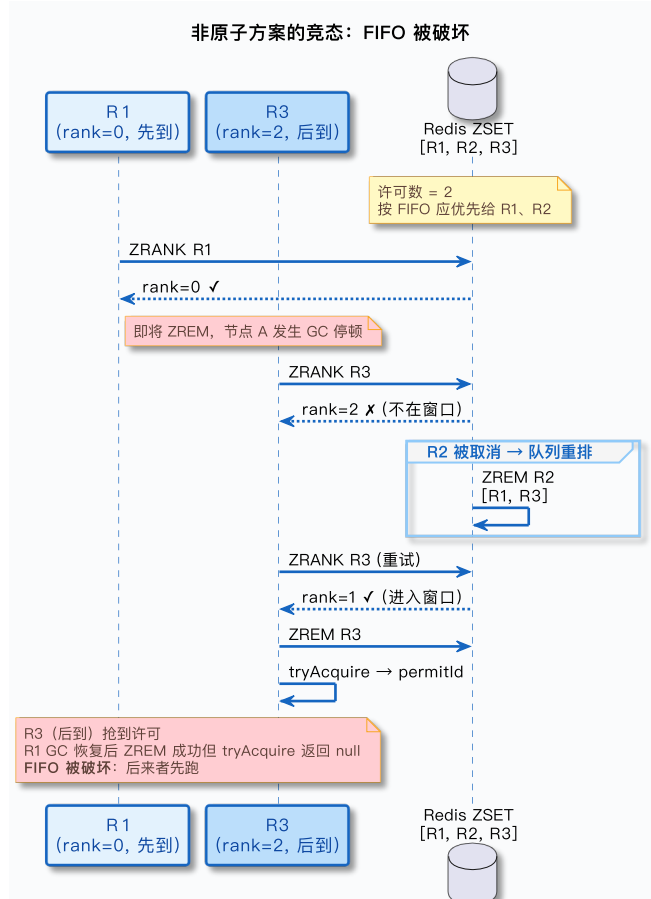
ZREM queueKey requestId 出队 + DEL entryPrefix..requestId 删除对应的 entry 标记。两步都在 Lua 内完成，避免被后续的 claim 扫描误判

返回 {1, score}: 第一个 1 表示成功，第二个是原始 score

整个脚本的语义升级了：不再是简单的 ZRANK 查排名，而是扫描队头窗口、过滤掉僵尸、按存活排名判断是否在窗口内。如果当前请求在存活成员的前 N 个 (N = 可用许可数)，就把它从队列和 entry 标记里一起移除，准备去抢许可。否则不做任何动作 (僵尸的清理是副作用，不影响当前请求的判定)。

3.3 为什么必须 Lua 原子

考虑一下，如果不用 Lua，把 ZRANK 和 ZREM 拆成两个 Redis 命令分两次 RTT，会发生什么？



非原子情况下，ZRANK 和 ZREM 两步之间有时间窗口。在这个窗口内，队列可能因为其他请求被取消而发生排名变化，导致后来的请求在排名前移后抢先 claim 成功，而先来的请求因为 GC 停顿或网络延迟错过了许可。最终 FIFO 顺序被破坏——后到的反而先跑了。

Lua 脚本在 Redis 单线程执行环境下保证原子性：从遍历 ZRANGE 到 ZREM 是一个不可分的事务单元。同一时刻只有一个 Lua 在跑，其他节点的 Lua 必须排队等。这样就不可能出现「两个请求同时看到自己 liveRank=0」的情况——一个先跑完 ZREM 之后，另一个再跑遍历时看到的队列已经不包含前一个了，但许可数是动态的，最终能 acquire 成功的还是受信量限制。

新版 Lua 脚本比旧版多了僵尸清理，原子性更加重要——如果 EXISTS 检查和 ZREM 清理不在同一个原子操作里，两个节点可能同时看到同一个僵尸条目，一个清理后另一个误以为没清理过，导致队列状态不一致。

回到在线教育公司：10 个请求几乎同时打到两台机器，每台机器跑各自的 tryAcquireIfReady。Redis 单线程串行执行 Lua 脚本，第 1 个 Lua 用 ZRANGE 取出队头窗口，遍历时看到 R1 的 entry 标记存在 (liveRank=0)，满足 liveRank < maxRank(3)，ZREM R1 + DEL entry:R1 后队列变成 [R2, R3, ..., R10] 共 9 个；第 2 个 Lua 是 R2，同样扫描队头发现自己 liveRank=0，ZREM R2；第 3 个 Lua 是 R3，liveRank=0，ZREM R3。前 3 个都成功 claim。

第 4 个 Lua 跑过来——扫描队头窗口看到 R4 的 liveRank=0 (R1/R2/R3 都不在队列里了)，看起来还能 claim。但传入的 maxRank 已经是阶段 2 查到的当时的可用许可数。如果阶段 2 查时许可还是 3 但阶段 3 跑 Lua 时其实只剩 0 (被 R1/R2/R3 拿走了)，R4 的 Lua claim 会成功但阶段 4 tryAcquire 会失败——这就是 claim 成功但 acquire 失败的兜底场景。

4. 阶段 4：tryAcquirePermit 拿真实许可

```
private String tryAcquirePermit() {
    RPermitExpirableSemaphore sem = redissonClient.getPermitExpirableSemaphore(semaphoreKey);
    try {
        return sem.tryAcquire(0, leaseSecondsSupplier.getAsInt(), TimeUnit.SECONDS);
    } catch (InterruptedException ex) {
        Thread.currentThread().interrupt();
        return null;
    }
}
```

4.1 tryAcquire(0, lease, SECONDS) 三个参数

waitTime=0：不阻塞。立刻尝试拿一个许可，拿不到马上返回 null。这里必须是 0——我们已经在阶段 3 通过 Lua claim 了队头位置，是不是真能拿到许可由信号量当前状态决定，不应该让线程阻塞等待

lease=600：许可的租约时间（秒）。一旦拿到，许可绑定一个租约——600 秒内调用方必须主动调 release，否则 Redis 自动回收

timeUnit=SECONDS: 时间单位

你可能注意到这里没有 `trySetPermits()` 调用。`FairDistributedRateLimiter` 在 Spring 初始化阶段由 `ChatRateLimiterConfig` 配置 `initMethod = "start"`, `start()` 里一次性初始化信号量许可数并订阅 Pub/Sub:

```
@Bean(initMethod = "start", destroyMethod = "stop")
public FairDistributedRateLimiter chatRateLimiter(RedissonClient redissonClient,
                                                  RAGRateLimitProperties rateLimitProperties) {

    return new FairDistributedRateLimiter(
        CHAT_LIMITER_NAME,
        redissonClient,
        rateLimitProperties::getGlobalMaxConcurrent,
        rateLimitProperties::getGlobalLeaseSeconds,
        rateLimitProperties::getGlobalPollIntervalMs
    );
}
```

```
public void start() {
    redissonClient.getPermitExpirableSemaphore(semaphoreKey)
        .trySetPermits(maxPermitsSupplier.getAsInt());
    // ... 订阅 Pub/Sub ...
}
```

`trySetPermits()` 本身是幂等的 (Redis 里已有许可数就跳过), 放在 `start()` 避免了每个 poller 每轮抢占都多一次 Redis 往返。如果 Redis key 被手动删除导致信号量丢失, 重启应用即可恢复。

4.2 claim 成功但 acquire 失败的兜底

```
String permitId = tryAcquirePermit();
if (permitId == null) {
    setEntryMarker(ticket.requestId,
        Math.max(1, ticket.deadline - System.currentTimeMillis()));
    RScoredSortedSet<String> queue = redissonClient
        .getScoredSortedSet(queueKey, StringCodec.INSTANCE);
    queue.add(claimedScore, ticket.requestId);
    publishQueueNotify();
    if (!ticket.isPending()) {
        queue.remove(ticket.requestId);
        deleteEntryMarker(ticket.requestId);
    }
    return false;
}
```

如果 `tryAcquire` 返回 `null` (许可被抢光), 说明阶段 3 看到的可用许可数已经过期了。处理方式是:

`setEntryMarker(...)`: 重新写入 entry 存活标记 (阶段 3 的 Lua 脚本已经把旧标记 DEL 了), TTL 为剩余等待时间。如果不重新写标记, 重入队后 Lua 下一次扫描会因为找不到 entry key 而把这个条目当僵尸清理掉

`queue.add(claimedScore, requestId)`: 用 `claimIfReady` 返回的原始 score 重新入队, 保留原来的排队位次——你本来排第 0 名, 许可被别人抢走了, 重入队后还是第 0 名, 不会被丢到队尾

`publishQueueNotify()`: 广播通知, 让其他等待者再来一轮抢占

重入队之后还要再查一次 `ticket.isPending()`: 因为 `claimIfReady` 的 `ZREM` 已经把你从队列移除了, `cancel()` 的 `cleanup()` 此时 `remove` 是空操作。如果重入队后发现 `ticket` 已经被取消或超时, 必须自己把刚加回去的条目和 entry 标记都移除, 否则会产生一个僵尸条目占据队头窗口

回到在线教育公司: 如果两台机器同时进入阶段 4 抢最后一个许可, 只有一个能成功, 另一个进入「按原位重新入队 + 广播」分支, 保留自己的队头优先级, 等下次有许可释放时再跟着所有等待者一起抢。

5. 阶段后处理: 再次状态检查 + `ticket.grant()`

5.1 拿到 permit 后再查一次状态


```

if (!ticket.isPending()) {
    releasePermitQuietly(permitId);
    publishQueueNotify();
    return false;
}

```

阶段 4 拿到 permit 后，**再查一次** ticket 状态。

为什么要再查？阶段 1 检查到现在中间过了三步 Redis 操作（availablePermits、claim Lua、tryAcquire），少说几十毫秒。这段时间里前端可能已经断连，ticket.cancel() 已经被触发，状态已经从 PENDING 变成了 CANCELLED。这时候如果不查状态直接执行业务，就把许可发给一个已经不存在的连接了——LLM 流照样跑、token 照样烧、SSE 照样推，但前端早就关掉浏览器走人了。

releasePermitQuietly 安静释放许可（忽略已过期等异常）：

```

private void releasePermitQuietly(String permitId) {
    try {
        redissonClient.getPermitExpirableSemaphore(semaphoreKey).release(permitId);
    } catch (Exception ex) {
        log.debug("{} 释放 permit 失败（可能已过期）：{}", name, ex.getMessage());
    }
}

```

5.2 ticket.grant() 提交到线程池

```

publishQueueNotify();
return ticket.grant(permitId);

```

确认许可有效后，先 publishQueueNotify() 广播——你可能会问，我自己刚拿到许可，为啥要广播？因为还有其他等待者在轮询，他们需要知道现在有变化（虽然你拿走了一个，但可能还有许可剩；或者后续会有别人释放）。广播让等待者更快进入下一轮抢占。

然后调 ticket.grant(permitId)，这是 Ticket 状态机的核心方法：

```

boolean grant(String permitId) {
    permitRef.set(permitId);
    if (!state.compareAndSet(State.PENDING, State.GRANTED)) {
        if (permitRef.compareAndSet(permitId, null)) {
            releasePermitQuietly(permitId);
            publishQueueNotify();
        }
        return false;
    }
    unregisterFromNotifier();
    cancelFutureQuietly();
    Runnable wrapped = () -> {
        try {
            req.onAcquired().run();
        } finally {
            releaseHeldPermit();
        }
    };
    try {
        req.onAcquiredExecutor().execute(wrapped);
        return true;
    } catch (RejectedExecutionException ex) {
        log.warn("{} onAcquired 提交失败，降级为 timeout 拒绝路径", name, ex);
        releaseHeldPermit();
        cleanup();
        submitSafely(req.onTimeout(), "onTimeout(fallback)");
        return false;
    }
}

```

grant() 干了什么？逐步拆解：

- 1. 先设 permitRef，再 CAS 状态：顺序很关键。先把 permitId 存进 permitRef，再 CAS PENDING → GRANTED。如果 CAS 失败（被 cancel() 或 timeout() 抢先），cancel() 的 cleanup() 能看到 permitRef 里的值并释放它——避免许可泄漏
- 2. CAS 失败的 double-check：permitRef.compareAndSet(permitId, null) 再试一次，防止 cancel() 没来得及释放的情况。CAS 保证许可只释放一次
- 3. CAS 成功后注销 poller + 取消定时器：不再需要轮询抢占了
- 4. 用 try/finally 包装 onAcquired：这是新设计的核心——把 req.onAcquired() 包在 try/finally 里，finally 调 releaseHeldPermit() 释放许可。许可的释放跟着业务执行的生命周期走，不管业务正常结束还是抛异常，许可一定会被还回去。这也是为什么 cancel() 的 cleanup() 在 GRANTED 状态下不释放许可——许可已经由这个包装接管了
- 5. 提交到 onAcquiredExecutor（即 chatEntryExecutor）：真正执行包装后的 streamChat lambda

chatEntryExecutor.execute() 把业务提交到专用线程池：

- 不在当前线程执行：当前线程可能是 Tomcat 的 NIO 线程或排队相关的调度线程，跑 30 秒到 1 分钟的业务会阻塞它，影响其他请求
- 不在 SseEmitter 自带线程：SseEmitter 没有自带的执行线程
- 专用池资源隔离：chatEntryExecutor 是为 RAG 业务专门配置的线程池，核心线程数和最大线程数都设为 maxConcurrent（与信号量许可数一致），配合 SynchronousQueue（无缓冲队列）实现精确控制——有空闲线程就立刻执行，没有就直接拒绝

5.3 提交失败的降级

```
    } catch (RejectedExecutionException ex) {
        log.warn("[{}] onAcquired 提交失败，降级为 timeout 拒绝路径", name, ex);
        releaseHeldPermit();
        cleanup();
        submitSafely(req.onTimeout(), "onTimeout(fallback)");
        return false;
    }
```

如果线程池满了，AbortPolicy 会抛 RejectedExecutionException。此时包装的 wrapped Runnable 还没跑，finally 里的 releaseHeldPermit() 不会被触发，所以这里必须手动调 releaseHeldPermit() 释放许可。然后 cleanup() 清理队列和标记，最后走 onTimeout 回调（在 Chat 场景里就是 handleReject，给用户推系统繁忙消息）。这里不尝试重新入队给第二次机会，而是直接走拒绝路径。原因是：如果线程池都满了，说明系统已经过载，重新入队抢到许可也还是会被拒绝，不如直接给用户一个明确的反馈。这种情况在生产中应该极少发生（线程池大小和信号量许可数一致，理论上不会超），更多是个保险。如果频繁出现说明 chatEntryExecutor 配置和信号量许可数不匹配。

ZSET vs List：为什么要选 ZSET

前面反复提到 ZSET，但没仔细对比过 List。你可能会觉得 List 更直观——LPUSH 入队 RPOP 出队，标准的 FIFO 队列，干嘛搞这么复杂？来看 List 在排队限流场景下的几个硬伤。

维度	List	ZSET
任意位置查询	不支持（要 LRANGE 拉全表自己找）	ZRANK key member 直接返回排名
任意位置删除	不支持（LREM 按 value 找，O(N)）	ZREM key member 直接删除
顺序保证	严格按入队顺序	按 score 排序（可控）
队头窗口判断	不支持「我是不是排在前 N 个」	ZRANK < N 一行搞定
同 score 处理	不适用	按 lex 顺序（用单调 seq 避免冲突）
原子操作	单条命令原子	单条命令原子 + Lua 复合操作

具体到排队限流，最关键的是两点：

- 第一，「队头窗口判断」。当前可用许可数是 N（动态变化），需要判断当前请求是不是排在队头前 N 个。List 没有「排名」这个概念——你可以 LRANGE 0 N-1 把前 N 个拉出来，再遍历找自己。但这是 O(N) 操作，每次抢占都要做，性能差。ZSET 的 ZRANK 直接给排名，O(log N)，再和 N 比较就完事。
- 第二，「随时取消」。SSE 连接随时可能断，断了要从队列移除自己。List 的 LREM key count value 按值找的开销是 O(N)，整个队列的所有请求一起断，删除开销爆炸。ZSET 的 ZREM 内部用 skip list，O(log N)，规模大也能扛。

还有一个隐性好处——ZSET 的 score 给了「顺序控制权」。我们用 RAtomicLong 单调递增做 score，FIFO；如果未来要做「VIP 用户优先」，给 VIP 用户一个更小的 score 就行（甚至负数），普通用户排到 VIP 后面。List 没办法这么搞。

RSemaphore vs RPermitExpirableSemaphore：为什么要选 expirable 版本

Redisson 提供了两种信号量：RSemaphore 和 RPermitExpirableSemaphore。代码里用的是后者，不是没原因的。

维度	RSemaphore	RPermitExpirableSemaphore
许可标识	没有 ID，acquire/release 凭计数	每个许可有 permitId
租约 (lease)	不支持	tryAcquire(time, lease, unit) 自动绑定
崩溃恢复	不支持，许可永久泄漏	lease 到期自动回收
重复释放	计数加多了 Redis 信号量错乱	release(permitId) 不存在的 ID 安全失败
适用场景	短期持有、调用方稳定不会崩	长期持有、调用方可能崩、需兜底

RAG 场景必须用 expirable 版本，原因有三：

- 第一，**长期持有 + 进程可能崩**。许可可能被持有 30 秒到 1 分钟，业务跑到一半进程被 kill 了，许可来不及 release。普通 RSemaphore 的许可就永久泄漏了——没人释放，下一个请求也无法 acquire 出更多。一旦泄漏 N 次，集群总并发就降低 N，最严重时整个限流系统瘫痪（信号量永远没有许可可发）。RPermitExpirableSemaphore 的 lease 是 600 秒，崩溃后最多 10 分钟许可自动回收，**自我修复**。
- 第二，**permitId 防重复释放**。普通 RSemaphore 的 release() 只是计数 +1，调用方调几次加几次，逻辑错误会让信号量计数虚高，超出 max-concurrent 限制。expirable 版本要求 release(permitId) 传 ID，Redis 端校验该 ID 还在持有列表里才允许释放，重复释放会被识别并忽略。
- 第三，**和异步路径的契合**。下篇会讲到，调度轮询的回调在不同线程上执行，许可的持有方和释放方可能不是同一个线程。expirable 版本的 permitId 可以跨线程传递、跨节点存储（虽然这里没用到跨节点释放，但保留了灵活性），普通 RSemaphore 没法做到「定向释放」。

一句话总结选型理由：RAG 这种长耗时 + 进程崩了许可需要兜底 + 异步释放的场景，必须用 RPermitExpirableSemaphore。普通 RSemaphore 适合短期持有、调用方非常稳定的场景，比如几十毫秒的数据库连接限流。

小结与下一篇预告

本篇讲了排队限流的同步路径，把视角从单个请求拉远到整个集群，回答了「为什么需要 + 怎么入队 + 怎么立即抢占」。核心要点回顾：

- RAG 场景的特殊性决定方案选型**：长耗时 + 资源敏感 + 体验敏感，QPS 限流挡不住长连接，本地 Semaphore 在多机部署下管不住总并发，直接 HTTP 429 体验差
- 两层架构分离关注点**：FairDistributedRateLimiter 封装通用的分布式公平限流逻辑，ChatQueueLimiter 只做 SSE 业务编排——emitter 生命周期绑定、超时拒绝写记忆、推送 SSE 事件
- 核心抽象是「信号量 + 排队队列 + entry 存活标记」三数据结构**：RPermitExpirableSemaphore 管集群总坑位，ZSET 管等待者按什么顺序进，entry 标记防僵尸条目永久占据队头
- Ticket 状态机统一管理竞态**：PENDING → GRANTED / TIMED_OUT / CANCELLED 三个终态互斥，单次 CAS 保证业务回调最多触发一次，cleanup() 幂等清理资源（GRANTED 状态下不释放许可，由 grant() 的 try/finally 包装接管）
- 必须用 RPermitExpirableSemaphore 而不是 RSemaphore**：lease 自动回收防泄漏，permitId 防重复释放
- 必须用 ZSET 而不是 List**：ZRANK 查任意位置 + ZREM 删任意成员，对应「队头窗口判断」和「随时取消」两个核心需求
- score 用单调 seq 不用时间戳**：避免毫秒级冲突破坏 FIFO 顺序
- Lua 脚本保证 claim 原子性并顺带清理僵尸**：遍历队头窗口、过滤掉 entry 标记已过期的僵尸条目、按存活排名判断是否在窗口内，整个过程必须原子，否则多节点会出现竞态
- 两阶段抢占：先 claim 再 acquire**：claim 成功只是占了队头位置，最终拿不拿到许可由信号量决定，两步都成功才算抢到；claim 成功但 acquire 失败时按原 score 重入队并补写 entry 标记，保留 FIFO 位次
- streamChat 直接委托 ChatQueueLimiter**：业务方法用 Runnable onAcquire 把方法调用包装成可异步执行的对象，由排队器决定何时、在哪个线程上执行

但本篇只讲了一半故事——**只有快速通道走通了**。

回到在线教育公司的促销现场：10 个请求同时来，前 3 个走完 tryAcquireIfReady 拿到许可执行业务，剩下 7 个进入队列等待。本篇没回答的问题是：

- 抢不到许可的 7 个请求该怎么办？要让前端傻等吗？**——不能傻等，得有一个调度循环周期性地重新尝试抢占
- 节点 A 上的请求等着节点 B 释放许可，节点 A 怎么知道许可释放了？**——靠 Pub/Sub 跨集群广播，但要防惊群（一次释放唤醒所有节点全员暴打 Redis）

等了 20 秒还没抢到该怎么给用户一个交代？——要走超时拒绝路径，不能让 SSE 连接挂着不响应。被拒的请求还要把消息写入会话记忆，否则用户下一句追问会失去上下文

如果业务跑完了忘记释放许可，许可永久泄漏怎么办？——靠 `RPermitExpirableSemaphore` 的 `lease` 兜底自动回收

这些问题构成了下一篇的全部内容——调度轮询的 `deadline` 机制、Pub/Sub 跨集群广播唤醒、`PollNotifier` 防惊群设计、超时拒绝写入会话记忆、许可释放与 `lease` 兜底。这套机制是整个排队限流看起来很简单实际很复杂的地方，集中在异步路径这一块。异步等待路径（`scheduleQueuePoll`）、`PollNotifier`、`Ticket.timeout()` 等逻辑都住在 `FairDistributedRateLimiter` 里，而超时拒绝的业务处理（写会话记忆、推 SSE 事件）通过 `AcquireRequest.onTimeout` 回调交给 `ChatQueueLimiter`。

到第 18 篇，AI 知识问答 18 篇正式收束。

